

Matthew Bernardino
Professor Anshul
ECPE 136
12 February 2026

Lab Assignment 4 - 8x8 Array Multiplier

Objective:

- The objective of this lab assignment is to create a 8x8 array multiplier. With the creation of this 8x8 array multiplier we have to learn about how to implement it in the way that it would be broken down, rather than a simple multiplication operation. We are multiplying two 8-bit binary inputs to make a 16-bit result. We are using partial products, AND gates, and learning how each partial product gets shifted through a process to get the final result through multi-stage addition and carry propagation.

Procedures:

1.0 - Reuse half_adder.sv, full-adder.sv, and 4-bitRCA.sv

- We reuse the following .sv files since we are using modular design implementation. Since we had already used these to create and verify with a previous lab assignment. We only need to create new scripts that will use the already known work to complete the array 8x8 multiplier. The use of a half adder, full adder, and 4 bit ripple carry adder are all used to pass along a chain to complete the multiplication. This teaches to be more efficient and focus on new scripts that need implementation and no knowledge of it working already.

2.0 - Create rca_16bit.sv

- We create a 16 bit ripple carry adder using the 4 bit ripple carry adder we have already made by instantiating 4 blocks to make 16 bits. The 16 carry ripple adder is in charge of adding two shifted partial product rows.

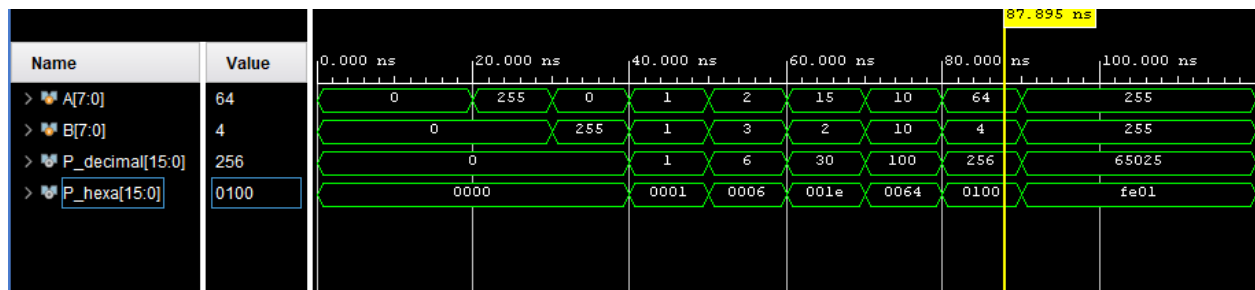
3.0 - Create array_multiplier8x8.sv

- We create an 8 by 8 array multiplier as the top level that uses all the previous files to do the multiplication itself. The other files are in charge of doing addition, the multiplication aspect needs to be added. The top level generates the partial product through AND gates and aligns each row to the correct bit position by shifting. The 4 block 16 bit ripple carry adder adds the 4 bits together to a final 16 bit result. This file also indicates the inputs and outputs that will be used for testbenching.

4.0 - Create array_mult8x8_tb.sv

- We create a test bench to verify that the top level array_multiplier8x8.sv completes the multiplication properly. Since this is the top level, we use the inputs A, B, and the output P to filter through two 8-bit values that will multiply to result in 16-bit output P. Since it is simulating multiplication, we can give values we know to verify multiplication.

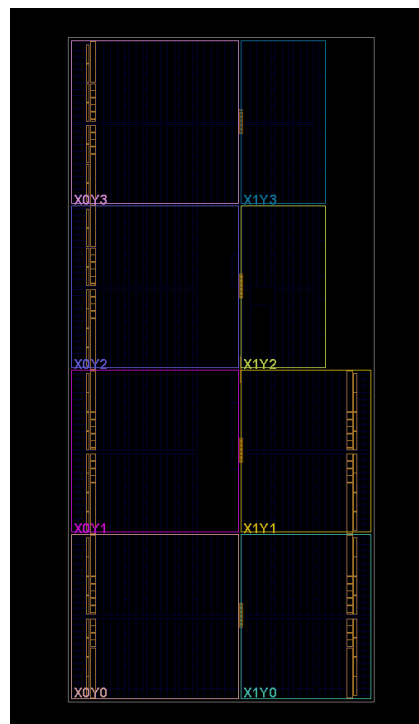
Observations & Results:



8 x 8 Multiplication Waveform

Multiplication Waveform Observations:

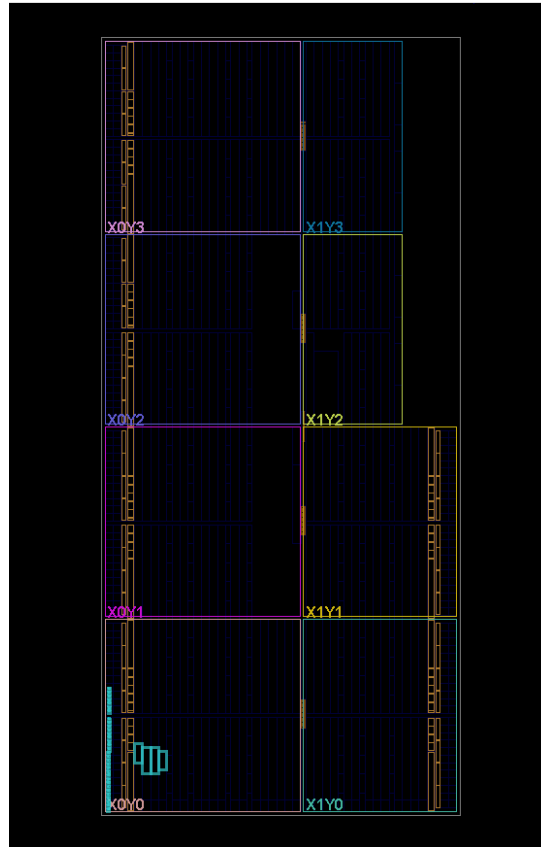
- Looking at the waveform, I was able to observe that all the multiplication test done were correct. I chose very simple values to test and then the absolute max value to be able to multiply as a 16 bit result. When doing easy multiplication like $2 \times 3 = 6$. I created both outputs to display the hexadecimal and decimal values for easy reading and comparison to make sure it's multiplying correctly. When doing the max value, to verify it will correctly do high value multiplication.



Synthesized Design

Synthesized Design Observation:

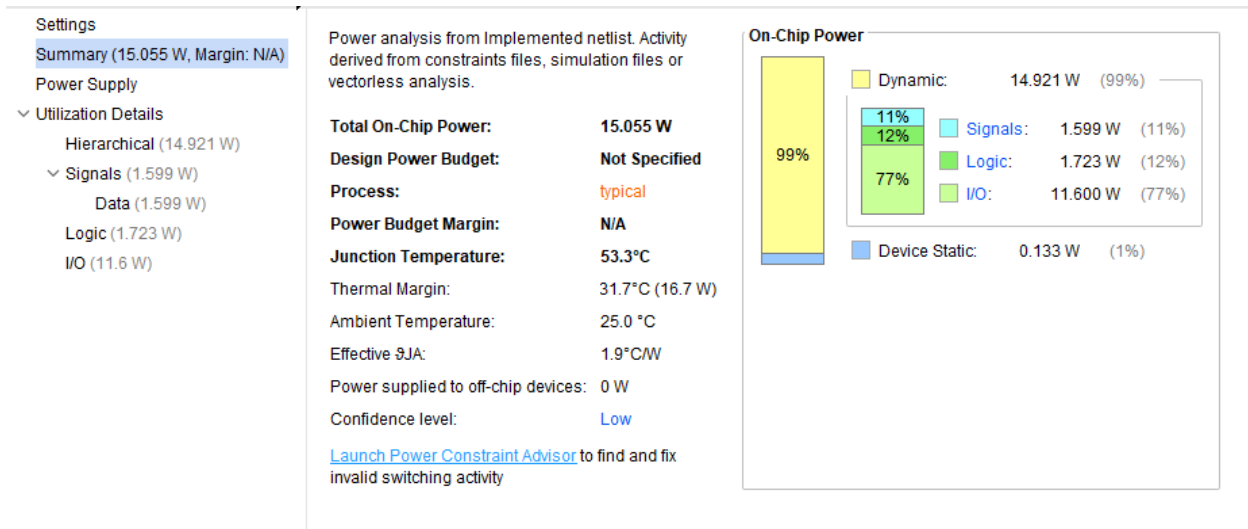
- The footprint is very small and does not indicate any heavy loads which means our design is not resource heavy. Since we are using straight forward simple design, there is not a lot of complexity. Instead we repeat a lot of adding, AND, shifting, etc so the image shows a similar repetitive structure that the code/hardware is doing.



Implementation Design

Implementation Design Observation:

- Within the implementation I can see that in the bottom left quadrant labeled X0Y0, the design fits within the region and the logic is small. There is repetition representing the AND rows and repeating the adders.



Power Report

Power Report Observations:

- Total on-chip power is 15.055 Watts.
- Dynamic Power: 14.921 Watts
- Static Power: 0.133 Watts
- I/O: 11.600 W (77%)
- Logic: 1.723 W (12%)
- Signals: 1.599 W (11%)
- When observing I noticed that the confidence level is low, this is related to the switching activity as I/O. While it is a simulation, there is some static/leakage of 1%. This could be transistor leakage current, or baseline power needed to power the chip in idle.

Conclusion:

- This lab assignment has taught how to create a 8 by 8 array multiplier in system verilog. Using a chain of multiple scripts that make up the multiplication on a top level. The goal is to learn and become more confident in modular design and familiarity with partial product formation, bit alignment, and carry propagation. With this I learned how to download a power report, create a synthesized and implementation design with native vivado tools. This allows us to visually see the code in a hardware context and see its effects on a FPGA.

Appendix

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Create Date: 02/17/2026 02:48:13 PM
// Module Name: array_multiplier8x8
/////////////////////////////////////////////////////////////////
// Matthew Bernardino
// professor Anshul
// ECPE 136
// 8x8 Array Multiplier

module array_multiplier8x8(
    input logic [7:0] A, // multiplicand
    input logic [7:0] B, // multiplier
    output logic [15:0] P // product
);

    logic [7:0] partial_p [0:7]; // and gate matrix array
    logic [15:0] partialp_shift [0:7]; // indented rows, shift left by x to multiply by 2
    logic [15:0] sum_a [0:7]; // addition, the rca_16 blocks the final 16bit product is stored
    to P

    // this loop does all the math in parallel instead of sequentially, since all comparisons are
    independent.
    // reference: https://www.systemverilog.io/verification/generate/
    // need to use partial_p as an array of 8 bits to individually take each bit from an 8 bit binary
    to AND gate
    // A is an 8bit binary number and B is also 8 bit binary but each individual bit from B gets
    AND with all of A
    genvar i; // create i variable for for loop
    generate // create for loop
        for (i = 0; i < 8; i++) begin : partial_p_AND // 8 increments for 8 bits, increment i with
        each new AND comparison
            assign partial_p[i] = A & {8{B[i]}}; // makes each B bit individually AND With all of
            A 8bit
        end //B is either 11111111 or 00000000 for each comparison
    endgenerate

    // partialp_shift loop. Another array of 8 bits.
    // need to shift left 8 bit partial_p to 16 bits, we shift based on the partial_p[k]
    genvar k; // make variable for for loop
```

```

generate
    for (k = 0; k < 8; k++) begin : product_shift
        assign partialp_shift[k] = {8'b0, partial_p[k]} << k; // make the 8bit partial_p as 16 bits
    end
endgenerate

// for proper shifting
// << k shifts left by 2^k. as k increments the shift is increased.

assign sum_a[0] = partialp_shift[0]; // there is no addition for the first row[0] so we add it in directly
// needs another value to add to it
// the actual addition of 7 adders and 8 rows to make one final result
genvar j; // create variable for for loop
generate
    for (j = 1; j < 8; j++) begin : sum_addition // for loop starts from
        rca_16bit final_adder ( // instantiate from rca_16bit.sv to use 4 bit block adders
            .a(sum_a[j-1]), // since we start sum_a[0] outside loop, we need to j-1 to go back
            .b(partialp_shift[j]), // next shifted partial product to add
            .s(sum_a[j]) // updates the 16 bit sum as it iterates with shifted partial products
        );
    end
endgenerate

// sum_a[7] has the final 16 bit output and we need to assign it to P since its the output wire
assign P = sum_a[7];

endmodule

```

array_multiplier8x8.sv

```

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////
// Create Date: 02/18/2026 10:47:21 AM
// Module Name: array_mult8x8_tb
//////////////////////////////////////////////////////////////////
// Matthew Bernardino
// professor Anshul
// ECPE 136
// 8x8 Array Multiplier testbench

module array_mult8x8_tb();
    // variable setup and output setup
    reg [7:0] A;
    reg [7:0] B;
    wire [15:0] P_decimal;
    wire [15:0] P_hexa;

    // instantiate inputs/outputs to top level
    array_multiplier8x8 mult8x8_inst(.A(A), .B(B), .P(P_hexa));
    // i want to show both hexa and decimal values in waveform for ease of reading
    assign P_decimal = P_hexa;

    initial begin
        // start both inputs at 0
        A = 8'h00;
        B = 8'h00;
        #10; // delay

        // inputs both 0 should have output 0 or 0x0000
        A = 8'h00;
        B = 8'h00;
        #10; // delay

        // 255 x 0 should output 0 or 0x0000
        A = 8'hFF;
        B = 8'h00;
        #10; // delay

        // 0 x 255 should output 0 or 0x0000
        A = 8'h00;
        B = 8'hFF;
        #10; // delay

        // 1 x 1 should output 1 or 0x0001
        A = 8'h01;

```

```

    B = 8'h01;
    #10; // delay

    // 2 x 3 should output 6 or 0x0006
    A = 8'h02;
    B = 8'h03;
    #10; // delay

    // 15 x 2 should output 30 or 0x001E
    A = 8'h0F;
    B = 8'h02;
    #10; // delay

    // 10 x 10 should output 100 or 0x0064
    A = 8'h0A;
    B = 8'h0A;
    #10; // delay

    // 64 x 4 should output 256 0x0100
    A = 8'h40;
    B = 8'h04;
    #10;

    // 255 x 255 should output 65025 or 0xFE01
    A = 8'hFF;
    B = 8'hFF;
    #10; // delay

    #20 $finish;

end
// Monitor output, added the same output to have both hexadecimal and decimal to compare
easily
always @(A, B, P_decimal, P_hexa)
    $monitor("Time=%0t ns, A=%0d, B=%0d, P_decimal=%0d,P_hexa=0x%h", $time, A, B,
P_decimal, P_hexa);

endmodule

```

array_mult8x8_tb.sv


```

`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 02/17/2026 02:56:49 PM
// Design Name:
// Module Name: rca_16bit
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
/////////////////////////////////////////////////////////////////
// Matthew Bernardino
// Professor Anshul
// ECPE 136
// Description: Use 4-bit RCA to make 4 blocks to make a 16 bit rca

module rca_16bit(
    // make 4 bit variables using array
    input logic [15:0] a, // 16 bit inputs
    input logic [15:0] b,

    output logic [15:0] s // 16 bit output sum
);
    logic cout0; // carry out bits 0–3
    logic cout1; // carry out bits 4–7
    logic cout2; // carry out bits 8–11
    logic cout3; // carry out bits 12–15

    RCA_4bit B0(
        .a(a[3:0]),
        .b(b[3:0]),
        .ci(1'b0), // no carry in the beginning
        .s(s[3:0]), // 4 bit part of sum
        .co(cout0) // carry out goes to next block
    );

    RCA_4bit B1(

```

```

        .a(a[7:4]),
        .b(b[7:4]),
        .ci(cout0), // carry from previous block
        .s(s[7:4]), // 4 bit part of sum
        .co(cout1) // carry out goes to next block
    );

    RCA_4bit B2(
        .a(a[11:8]),
        .b(b[11:8]),
        .ci(cout1), // carry from previous block
        .s(s[11:8]), // 4 bit part of sum
        .co(cout2) // carry out goes to next block
    );

    RCA_4bit B3(
        .a(a[15:12]),
        .b(b[15:12]),
        .ci(cout2), // carry from previous block
        .s(s[15:12]), // 4 bit part of sum
        .co(cout3) // final carry out
    );

endmodule

```

rca_16bit.sv

```
// Name: Matthew Bernardino
// Date: 2/18/2026
// Class: ECPE 136
// Professor: Anshul
// Description: 4-bit Ripple Carry Adder - Use 4 Full adders to add two 4 bit numbers
```

```
module RCA_4bit(
```

```
// make 4 bit variables using array
    input logic [3:0] a,
    input logic [3:0] b,
    input logic ci, // carry in handler
```

```
    output logic [3:0] s, // 4 bit sum
    output logic co // carry output
```

```
);
```

```
// setup wire connections of full adders
// set carry outs for pass full adder to full adder
logic co0_to_ci1;
logic co1_to_ci2;
logic co2_to_ci3;
```

```
full_adder FA0 (
    .a(a[0]), // connect full adder script to current 4 bit array
    .b(b[0]), // first a and b inputs
    .ci(ci), // first carry input
    .s(s[0]), // makes bit 0 sum
    .co(co0_to_ci1) // hands carry out to next full adder
);
```

```
full_adder FA1 (
    .a(a[1]), // second a and b inputs
    .b(b[1]),
    .ci(co0_to_ci1), // carry in from previous full adder
    .s(s[1]), // makes bit 1 sum
    .co(co1_to_ci2) // hands carry out to next full adder
);
```

```
full_adder FA2 (
    .a(a[2]), // third a and b inputs
    .b(b[2]),
    .ci(co1_to_ci2), // carry in from previous full adder
    .s(s[2]), // makes bit 3 sum
    .co(co2_to_ci3) // hands carry out to next full adder
```

```
);  
  
full_adder FA3 (  
    .a(a[3]), // third a and b inputs  
    .b(b[3]),  
    .ci(co2_to_ci3), // carry in from previous full adder  
    .s(s[3]), // makes bit 3 sum  
    .co(co) // has final carry output, if there is one  
);  
  
endmodule
```

4-bitRCA.sv

```
// Name: Matthew Bernardino
// Date: 2/18/2026
// Class: ECPE 136
// Professor: Anshul
// Description: full Adder - add three bits using two half adders with OR gate
```

```
module full_adder (
    input logic a, // input 1
    input logic b, // input 2
    input logic ci, // carry in from previous add
    output logic s, // output sum
    output logic co // output carry
);

    // setup for connecting two half adders together
    logic sum_half_adder1; // sum
    logic carry_half_adder1; // carry output from first half
    logic carry_half_adder2; // carry output from second half

    half_adder first_ha (
        // connect two half adders together
        .a(a),
        .b(b),
        .s(sum_half_adder1), // partial sum
        .carry(carry_half_adder1) // carry of a & b
    );

    // adds partial sum with carry in
    half_adder second_ha (
        .a(sum_half_adder1), // partial sum from first half adder
        .b(ci), // adds carry in
        .s(s), // final added output
        .carry(carry_half_adder2) // another carry
    );

    // combine both carry outputs with OR gate
    assign co = carry_half_adder1 | carry_half_adder2;

endmodule
```

full_adder.sv

```
// Name: Matthew Bernardino
// Date: 2/18/2026
// Class: ECPE 136
// Professor: Anshul
// Description: half Adder - add two bits to make a sum and carry

module half_adder(
    input logic a, // input 1
    input logic b, // input 2
    output logic s, // output sum
    output logic carry // output carry
);

    assign s = a ^ b; // need XOR for sum

    assign carry = a & b; // if both a and b are 1 , we carry it like in 1 + 1 = 1 carry

endmodule
```

half_adder.sv